Implementation of Firmware Based on Real-time Operating System for Small Size League Robot

1st Andrei Freiberger
Autonomous Computational Systems Lab
São José dos Campos, SP, Brazil
andrei.fr6@gmail.com

2nd Marcos R. O. A. Maximo
Autonomous Computational Systems Lab
Aeronautics Institute of Technology
São José dos Campos, SP, Brazil
mmaxmio@ita.br

3rd Denis Loubach
Computer Science Division
Aeronautics Institute of Technology
São José dos Campos, SP, Brazil
dloubach@ita.br

2024 Brazilian Symposium on Robotics (SBR), and 2024 Workshop on Robotics in Education (WRE) | 979-8-3503-9108-4/24/$31.00 ©2024 IEEE | DOI: 10.1109/SBR/WRE63066.2024.10837771

***Abstract*—This article presents the implementation of a real-time operating system (RTOS) on ITAndroids Small Size League (SSL) category robots, a significant advancement for the team's participation in the Robocup SSL. The transition to using the STM32H753BI microcontroller and FreeRTOS on the second-generation *Mainboard* enhances temporal predictability and determinism, crucial for real-time systems. This development allows more efficient task management, including motor control, kicker charging, telemetry transmission, and fault handling, leading to improved robot performance and reliability during matches. The detailed system description and task implementation showcase how the *Mainboard* efficiently handles various functions such as motor control in a constant rate, responsive kicking mechanisms, and real-time telemetry collection and transmission. The work contributes by proposing a real-time operating system for a robot designed to participate in the RoboCup Small Size League (SSL). Despite the implementation being dependent on our team's design decisions, the concepts presented here can be adapted by other teams.**

***Index Terms*—Real-Time Operating Systems, *FreeRTOS*, Firmware, *Small Size League*, Implementation**

I. Introduction

The Small Size League (SSL) is one of the robot soccer categories in the Robocup competition, in which ITAndroids team has participated annually [1]. Each of the team's SSL category robots executes external commands through a printed circuit board called *Mainboard*, which concentrates most of the robot's functions. The second generation of the *Mainboard* uses the STM32H753BI 32-bit Arm Cortex-M7 single core microcontroller (hard core) [2]. Such a microcontroller allows the use of an open source real-time operating system (RTOS), the FreeRTOS [3], as a middleware.

A real-time system must meet non-trivial time requirements, such as deadlines for completing a task. This results in greater temporal predictability of tasks and greater determinism in the system behavior [4]. Therefore, it is widely used in critical systems such as satellites [5], radars, defense systems and aircraft [4].

FreeRTOS is an open source real-time kernel, maintained by Real Time Engineers Ltd, aimed at applications on microcontrollers or small microprocessors [3]. This system manages multiple tasks with different priorities. The middleware itself provides several structures to perform interactions between tasks, resource management and synchronization. Some of these structures are: message queue, in which a data structure can be passed from one task to another; semaphore, a synchronization tool used to manage resource access by multiple tasks, operating as a counter to signal when resources are available or need to be released; mutex, a synchronization mechanism used to ensure that only one task can access a shared resource at a time, preventing concurrent access issues; timer, a software mechanism that triggers a callback function after a specified period, enabling timed execution of tasks or operations.

This article aims to present an implementation of a complex task model for SSL robots running on an ARM M7 microcontroller, using FreeRTOS as a real-time operating system.

The paper is structured as follows:

- **Section 2: Literature Review** - In this session, we discuss some work related to mobile robotics and real-time operating systems.
- **Section 3: System Description and Requirements** - In this section, we describe the hardware of the second-generation *Mainboard* and the system requirements for controlling and managing the different components of the robot.
- **Section 4: Task Model and Implementation** - We present the task model implemented using FreeRTOS, including the system configuration and the description of the individual tasks. We explain how each task is managed and its interaction with the real-time system.
- **Section 5: Results** - In this section, we discuss the results of the tests performed with the system, including the analysis of the execution times of the tasks and the effectiveness of the implementation during the bench tests and in the Robocup 2024 matches.
- **Section 6: Conclusion** - We close the article with an overview of the main results and conclusions about the implementation of the real-time operating system, and discuss possible improvements and future challenges.

II. Literature Review

Using RTOS in mobile robotics is relatively common. In the work of [6], they compare the use of general purpose operating system and real-time operating system in mobile robots running on Raspberry. In the work of [7], a control system

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:34:22 UTC from IEEE Xplore. Restrictions apply.

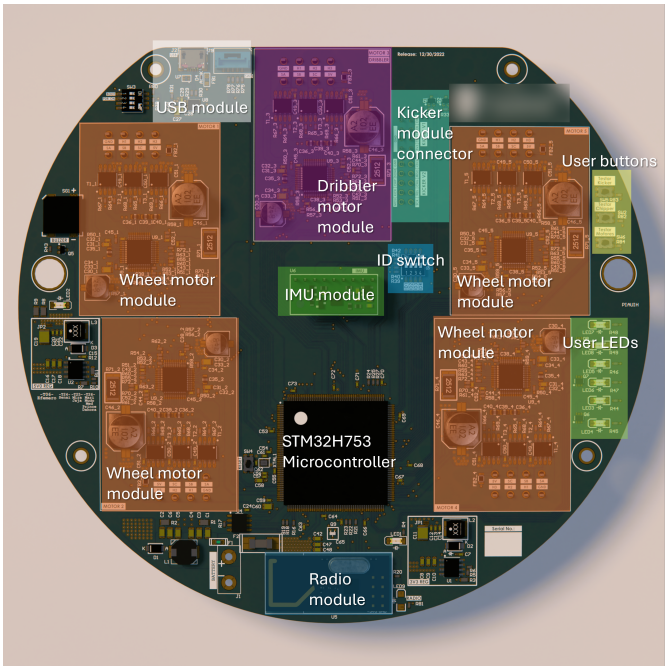


Fig. 1. Rendered photo of the second generation *Mainboard* printed circuit board with its main modules highlighted.

for swarm robots was implemented in ARM microcontroller using FreeRTOS as a base.

The use of FreeRTOS in SSL robot firmware is not new, the American team RoboJackets mentions its use in the 2016 TDP [8], but it was applied to a FPGA. The Canadian team ULtron uses freeRTOS on an ARM M4 running simply three threads [9]. An omnidirectional robotics application on ARM M4 microcontrollers was presented in [10]. In this work, threads for movement, communication, IMU processing, among others, were developed, similar to what is intended to be applied in our SSL robots.

III. System Description and Requirements

The second generation *Mainboard* can be seen in Figure 1. In the figure, the orange areas highlight the modules that drive the wheel motors; in dark blue, the module that performs radio frequency (RF) communication using an nRF24L01 radio [11]; in purple, the motor module of the Dribbler, an electromechanical device responsible for holding the ball close to the robot; in cyan, the interface between the *Mainboard* and the *Kickboard*, a printed circuit board that operates an electromechanical device responsible for executing the robot's kicks; in light blue, the dip switch that defines the robot's identifier (ID) and the radio channel to be used; in yellow, the buttons used to perform tests; in light green, LEDs indicating some of the robot's states in dark green, the Inertial Measurement Unit (IMU); and in white, the module that performs the USB interface and Serial Wire Debug (SWD) of the microcontroller with an external computer for debugging and programming.

A. Reception and generation of telecommands

Each SSL robot has an identifier (ID) that can be configured by the user via a dip switch. Since the robot's ID can be changed by the team during a match, the microcontroller must check the robot's ID every 1 s, which is a flexible deadline. A packet containing commands for the robots must be received via radio from the communication station, which sends messages approximately every 16 ms (60 Hz). Since it is a broadcast communication and each message has commands for three robots, the microcontroller must identify and extract its part of the message that refers to its ID, and thus obtain information on tangential, normal and angular speed, the command to perform a high or low kick, among other commands. The robot must respond to these commands, which can be delayed by a few milliseconds, since consecutive commands tend to be similar in most cases.

The *Mainboard* should also run debug tests to check the operation of the motors and kick. The choice of tests should be made by the debug buttons and may depend on the selected ID. These tests must emulate commands that would be received via radio.

B. Motors Control

From the wheel speed reference calculated from the speed information in the message and the wheel speed estimate, the discrete speed control calculation in a closed loop must be performed. The speed of the brushless motor is estimated by means of Hall sensors or encoders fixed to the motor's electromechanical assembly. The control calculation, using a proportional-integral controller, must occur at a sampling rate of 1 kHz (in the case of using encoders) or 200 Hz (in the case of using Hall sensors). The difference in sampling rates comes from the resolution of each hardware. Since Hall sensors have a very low resolution, a control rate of 1 kHz would cause the wheel speed estimate to become very noisy, which impairs the control action. Regardless of which hardware is used to perform the speed measurement, the sampling rate must be constant, since rates that vary over time tend to harm the performance of the discrete controller [12], i.e., the period between the execution of each iteration of the control is rigid. Therefore, compliance with this rate must be prioritized over the other tasks already mentioned.

C. Interface with the Kickboard

The *Kickboard* is a board connected to and controlled by the *Mainboard*. It is responsible for raising the battery voltage to 180 V and charging large electrolytic capacitors on command, and releasing the energy accumulated in these capacitors in solenoids responsible for the low and high kick. The *Kickboard* can identify when the capacitors are not charged and communicates this state via a General Porpuse Input Output (GPIO) pin to the *Mainboard*. In addition, capacitor voltage and hardware temperature measurements are performed on this board and communicated to the *Mainboard* via SPI protocol.

During the game, it is necessary to keep the capacitors charged, as a kick may be necessary at any time. To save

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:34:22 UTC from IEEE Xplore. Restrictions apply.

energy, the capacitors should be charged to the limit and then the charging should be stopped. Since the capacitors discharge slightly over time when charging is disabled, the charging status of the capacitors should be checked every 1 s, and if necessary, the charging should be re-enabled until the limit is reached again. This 1-second deadline is flexible.

For design reasons, the *Kickboard* should not release the capacitors' energy at the same time as they are being charged. The strength of the kick is determined by the amount of time the capacitors are discharged in the solenoid. For the reproducibility of the kicks and their respective strengths, this time period should be deterministic, making it a rigid deadline. The Kicker's cooldown time must be respected so that it does not get damaged. A conservative approach is a fixed time of 2.5 to 3 seconds, with at least 2 seconds of that time spent recharging the capacitors. During this cooldown time, kicks should not be performed even if commanded by accident.

D. Housekeeping

The *Mainboard* must monitor the internal temperatures, currents and voltages of the robot's electronic, electrical, and electromechanical systems, warn operators in case of malfunction, and even disable modules if necessary to maintain the integrity of the robot. Some important information is the motor currents, battery voltage, and solenoid temperature. This can be done at a low rate and with a flexible deadline.

E. Acquisition and transmission of telemetry

Finally, telemetry from all modules must be collected with their respective timestamps, and some selected ones must be formatted into a 32-byte message and sent via radio to the station at a predetermined rate. The selected information depends on the type of message to be sent. The type of message can be configured in the firmware or requested via radio, and can be a nominal message (used during games), a motor debug message (for motor controller debug), or a Kicker debug message. Each type of message has a specific sending rate, and cannot interfere with or delay any of the aforementioned functionalities. On the other hand, the messages to be send can be accumulated and delayed, since the packet always contains the timestamp of the telemetry.

IV. Task Model and Implementation

The embedded code was initially based on the code of the first generation of robots that was used in the last *Mainboard*. With the new board, the code was adapted to the new hardware, using the Hardware Abstraction Layer (HAL) drivers provided by STMicroelectronics and restructured in the form of tasks to meet the proposed requirements. Regarding FreeRTOS, the kernel was configured to be used through the CMSIS V2 abstraction layer. In addition, it was chosen to use preemptive scheduling, with the smallest time slice available for this kernel, of 1 ms, and with fixed priorities.

Based on the requirements, 8 tasks were created, whose characteristics can be seen in Table I.

TABLE I
System tasks and their respective characteristics

Task	Type	Rate	Priority	
`ID_UPDATE`	Periodic	0.5 Hz	Low	8
`RADIO_RECEIVE`	Sporadic	Interrupt (60 Hz)	Normal	24
`MOTOR_CONTROL`	Periodic	200 Hz or 1 kHz	High	40
`KICKER_CHARGE`	Periodic	1 Hz	Normal	24
`FIRE_KICKER`	Non periodic	—	Above Normal	32
`HOUSEKEEPING`	Periodic	1 Hz	Low	9
`FAULT_HANDLING`	Non periodic	—	Above High	41
`RADIO_TRANSMIT`	Periodic	10 Hz or 100 Hz	Below Normal	16

All tasks have an initialization step that is executed when creating the task, and a main loop. For a new iteration of the loop to be executed, each task needs to get a task specific semaphore that can be released in two possible ways: by a timer (in the case of periodic tasks) or by an event such as an interruption or from another task (in the case of aperiodic or sporadic tasks). Each periodic task has its own timer, whose callback function only releases the semaphore.

A. `ID_UPDATE` Task

To make the firmware more flexible especially during the development phase, all tasks are suspended during the task initialization phase, except the `ID_UPDATE` task. Only preselected tasks are passed to the "Ready" state after the reading of a valid ID ($0 < ID < 7$) from the DIP switch. Since the ID can be used in several tasks, it is a global variable protected by a mutex.

B. `RADIO_RECEIVE` Task

This task has a priority of 24, which represents the Normal priority level, and is considered sporadic, since some messages may be delayed or not sent by the central computer, which is not a real-time system. The configuration, message request and command execution in the nRF24L01 radio are done through serial communication with SPI protocol between the microcontroller and the radio. Whenever the radio receives a message, it generates a software interrupt that releases the semaphore that allow a new iteration of the task loop to run. This verification of the arrival of the message could be done by *polling*, a technique in which the status of a device or peripheral is regularly checked [13], but this constant verification would produce unnecessary processing time most of the time, as can be seen in the bench test result in Figure 2. The test was performed with the radio unitary module in two different ways, first using *polling* and then using the interrupt through the NRF_IRQ pin. The monitored pins were the interrupt pin and two of the pins used for SPI communication: the SCK and the MISO. It was observed that the use of interrupts generates a more efficient use of the processor.

Despite the improvement in processor efficiency, the use of interrupts in real-time software should be done with care, as they occur in a way that overrides the operating system, potentially increasing system latency and making the system less deterministic [14]. Interrupts impose an overhead on the system, as, in addition to their execution time, context switches consume time. However, given the 400 MHz clock frequency,

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:34:22 UTC from IEEE Xplore. Restrictions apply.



Fig. 2. Result of the bench test of interrupt pin monitoring (NRF_IRQ), SCK and MISO.

we observed that the time spent on context switching was negligible compared to the message reception rate (approximately every 16 ms).

Thus, the `RADIO_RECEIVE` receives the radio message and extracts the information relevant to the robot. The high-level speed information of the robot is processed through an inverse kinematics matrix that converts the robot's tangential, normal, and angular velocities into the wheels' angular velocities [15]. This speed information for each wheel is sent via a message queue to the motor control task, `MOTOR_CONTROL` task. The fire and power of the high or low kick is also received by radio, if the *Kickboard* is enabled and is not in the cooldown state, the necessary information is passed through the message queue and the semaphore of the `FIRE_KICKER` task is released.

C. `MOTOR_CONTROL` Task

The `MOTOR_CONTROL_TASK` has a high priority (40), one of the highest priorities in the system to ensure that it is executed at a constant rate. When executed, it checks for new speed references in the queue and updates the reference if possible.

To measure wheel speed using the Hall sensors, two pins from the motor's driver are used, called DIRO and TACHO. The TACHO pin combines the information from the three Hall sensors on the motor into just one signal. By counting the rising and falling edges n of the TACHO signal, and knowing the mechanical reduction ratio R, the number of motor poles P and the sampling period $T_s = 5$ ms, it is possible to estimate the wheel speed from

$$\bar{\omega} = \frac{n\pi}{3PRT_s}. \tag{1}$$

The TACHO pulse train edge count is performed by means of interruptions that increment a counter that is made available for the task. Since this increment is performed with few lines of code and the Hall sensors have a low resolution, that is, they generate few pulses to be counted and, therefore, few interruptions, the pulse count by means of interruption causes little temporal impact on the system. When encoders are implemented, the interruption rate will be very high and this solution will no longer be viable. For this purpose, the hardware timers of the STM32 microcontroller in Encoder Mode will be used, which handle the pulse count directly in hardware.

Based on the reference and the speed estimate, the control is executed and the interface with the motor is established. To activate the motor by means of the driver from the microcontroller, a single pulse width modulation (PWM) signal is required, which establishes the average voltage level applied to the coils, and a logical value that indicates the switching order to be executed, defining the direction of rotation of the motor. In the microcontroller, the PWM signals are generated by hardware, that is, it is not necessary to use the processor to maintain them, only to modify them, and the logic pins are carried out through GPIO.

D. `KICKER_CHARGE` Task

This task is periodic and has normal priority (24), since its deadlines are not rigid and the period between each execution is relatively long (1 second). At each iteration of this task, it checks the "NOT DONE" GPIO pin of the *Kickboard*, which indicates whether the capacitors are not fully charged. If so, and provided that the *Kickboard* is not in the fire or cooldown state, a signal is sent to the *Kickboard* to charge. If it is already charged to the limit, charging is disabled until the next check.

E. `FIRE_KICKER` Task

For a new iteration of this task to run, it is necessary to obtain the semaphore. Since kicking can be a critical event during the game, the priority of this task is slightly lower than the motor control task. During its execution, the *Kickboard* charging is initially turned off, entering the fire state, waiting a few milliseconds to ensure that the *Kickboard* has stopped charging, and then the kick is activated by GPIO for a time that depends on the Kick Power received in the message. After this time, the kick is interrupted, entering the cooldown state. Thus, at least half a second is waited until the charging is re-enabled. The cooldown state remains for up to 2 more seconds, enough time for the capacitors to charge. After the cooldown ends, a new semaphore release for kick is allowed. To prevent the *Kickboard* from kicking while it is charging, each of the "Fire" or "Charging" states needs to acquire a mutex to be executed.

F. `RADIO_TRANSMIT` Task

Since the transmission of messages via radio is not essential for the game, this task has a lower priority than the others. Each task has a message queue that is used to transmit the collected telemetry to the `RADIO_TRANSMIT` task. Since each task has a different rate, the `RADIO_TRANSMIT` task checks the number of messages in the queue, and if there are new messages, this information is used to update the telemetry struct. Depending on the type of message configured, the task formats the pre-selected telemetry in 32 bytes that will be sent via radio. To do this, it is necessary to reconfigure the nRF24L01 radio from receive mode to transmit mode, and then reconfigure it to receive mode again.

V. Results

The system was tested on a bench using a logic analyzer that monitored GPIO pins configured as microcontroller outputs that assumed a high logic level when the task began to be

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:34:22 UTC from IEEE Xplore. Restrictions apply.

executed and a low logic level when it was finished. From this, the execution time of the tasks and the interval between their multiple executions were measured in order to estimate the standard deviation of the measurements.

Since the *Mainboard* and *Kickboard* were still in its first iteration, some *hardware* elements did not work, preventing full testing of the aforementioned tasks, including battery voltage level monitoring, temperature and capacitor voltage from *Kickboard* and the use of encoders. Since Hall sensors were used, the 200 Hz control rate requirement was met instead of 1 kHz, measuring a frequency of 200 ± 1 Hz.

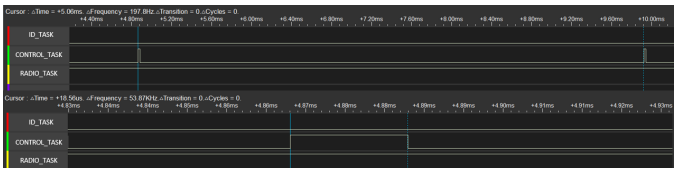


Fig. 3. Period between executions of the control task and task execution time for the case using the *Hall* sensors.

In some tests where the `RADIO_TRANSMIT` task was running at 100 Hz, it was noticed that many packets were not being delivered to the station. It was then observed that the task took 1.32 ms to complete. By removing the radio configuration and sending step, this time was reduced to a few microseconds as can be seen in Fig. 4. Thus, it was noticed that the radio reconfiguration time is very long, which forces us to use a lower transmission rate. If it is ever necessary to use feedback messages at high frequency (100 Hz or more, for example), it is interesting to have two radios on the board, one always configured for reception and the other always configured for transmission.

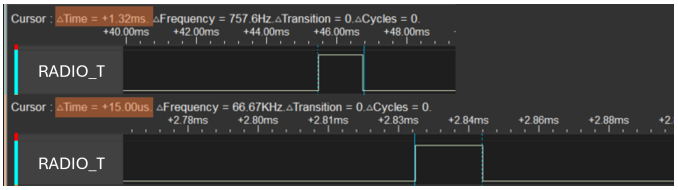


Fig. 4. Period between executions of the control task and task execution time for the case using the *Hall* sensors.

On average, using a *clock* of 400 MHz, the `MOTOR_CONTROL` task executes in 18 ± 1 µs, when performing the PI calculation for a single wheel. Performing the calculations for all 4 wheels, this value increases to 50 ± 1 µs. The `RADIO_RECEIVE` task, on the other hand, executes in $(29.0 \pm 1) \cdot 10^1$ ns, not counting the duration of the interrupt that releases its semaphore. Since the execution of each task is very short in temporal terms when compared to the period in which they are executed, it was observed that the system has no difficulty in meeting the timing requirements.

As a final test, the robot carrying the *Mainboard* with the implemented real-time system was tested during the Robocup 2024 matches, showing deterministic behavior, without unforeseen delays in command executions and ensuring the expected control rate, which enabled several victories for the team.

As a more qualitative operational result, it is important to note the difficulty of inserting new tasks in a way that does not significantly disrupt the temporal and functional behavior of the real-time operating system. Careless implementation of a new task can cause system delays, deadlocks, improper blocking of tasks, and complete system malfunction. During development, poorly planned implementations of the `RADIO_TRANSMIT` and `FIRE_KICKER` tasks caused delays in the `MOTOR_CONTROL` task, mainly due to the use of "critical sessions" in which preemption was disabled. On another occasion, a problem in the `RADIO_RECEIVE` task caused the packet received by the radio to not be removed by the microcontroller during the task execution. Thus, since a packet in the nRF24L01 buffer prevented other packets from being received, the interrupt that released the semaphore for the execution of the `RADIO_RECEIVE` task was never triggered again, which blocked the task forever.

VI. Conclusion

The implementation of the real-time operating system on the second-generation *Mainboard* for the Small Size League (SSL) robots has shown promising results in terms of meeting the stringent requirements of real-time operation and efficient task management. Utilizing the STM32H753BI microcontroller and FreeRTOS, the system successfully integrates multiple critical functionalities, such as motor control, kicker management, telecommand reception, and telemetry transmition.

The system's performance was thoroughly evaluated using a logic analyzer, which validated the timely execution of tasks and the system's capability to maintain rigid deadlines, especially for motor control and kicking operations. Despite the initial hardware limitations, such as the inability to fully test certain components like the battery voltage monitoring and the encoder usage, the tests conducted with Hall sensors demonstrated the system's ability to maintain the required control rate of 200 Hz with minimal deviation.

The implementation of interrupts for the `RADIO_RECEIVE` task significantly improved processor efficiency. However, challenges were encountered with the `RADIO_TRANSMIT` task, where the high reconfiguration time of the nRF24L01 radio module led to packet losses at higher transmission rates. This issue underscores the need for potential hardware improvements, such as the addition of a second radio module dedicated to transmission.

Overall, the integration of FreeRTOS with the second-generation *Mainboard* has enhanced the functionality and reliability of ITAndroids SSL robots, providing a robust platform for future developments, whose implementation is facilitated due to the RreeRTOS characteristic of dividing features into tasks, a new feature could simply be a new task or be implemented in an existing task. With the board corrections in a second iteration, the missing tasks will be tested and

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:34:22 UTC from IEEE Xplore. Restrictions apply.

validated. In addition, another future work is to add new tasks to deal with additional problems not addressed in this research, such as reading angular velocity and acceleration using the inertial measurement unit (IMU), that could be used in a navigation task; reading motor currents, which would allow better low-level control of the motors, among others. An important improvement would be to implement different test modes, which could aid in automatic and manual tuning of the low-level controller gains. Continued hardware optimization and comprehensive testing will further improve the system's performance, ensuring it meets the high demands of real-time robotic applications in competitive environments like RoboCup.

VII. Acknowledgements

Thanks to the National Development Council (CNPq), for providing scholarship to Andrei Freiberger to carry out this project. Marcos Maximo is partially funded by CNPq – National Research Council of Brazil through the grant 307525/2022-8. The authors would like to thank all ITAndroids members and the team's sponsors: Altium, Cenic, FieldPRO, ITAEx, Mathworks, Rapid, SIATT, Solidworks, STMicroelectronics, and Visiona.

References

[1] RoboCup Soccer Small Size League, "Robocup soccer small size league rules," 2024, accessed: 2024-08-04. [Online]. Available: https://robocup-ssl.github.io/ssl-rules/sslrules.html

[2] *STM32H753xI*, STMicroelectronics, April 2019, rev 7.

[3] (2023) Freertos™. https://www.freertos.org/index.html.

[4] R. S. de Oliveira, *Fundamentos dos Sistemas de Tempo Real*. Santa Catarina: Independently Published, 2020.

[5] J. J. Molini, S. K. Maimon, and P. H. Watson, "Real-time system scenarios," *Real-Time Systems Symposium*, 1990.

[6] M. A. Rabbah, N. Rabbah, H. Belhadaoui, and M. Rifi, "Designing middleware over real time operating system for mobile robot," in *Lecture Notes in Real-Time Intelligent Systems*, J. Mizera-Pietraszko, P. Pichappan, and L. Mohamed, Eds. Cham: Springer International Publishing, 2019, pp. 419–425.

[7] T. Docekal and Z. Slanina, "Control system based on freertos for data acquisition and distribution on swarm robotics platform," *2017 18th International Carpathian Control Conference (ICCC)*, pp. 434–439, 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:24615183

[8] S. C. B. I. e. a. Gabriel An, Justin Buchanan, "Robojackets robocup 2016 team description paper," 2016, accessed: 2024-08-05. [Online]. Available: https://wiki.robojackets.org/

[9] S. B. M. G.-L. e. a. Philippe Babin, Alexia Blanchard-Lapierre, "Ultron robocup 2017 team description paper," 2017. [Online]. Available: https://ssl.robocup.org/wp-content/uploads/2019/

[10] M. Musyafani, F. Ardilla, and M. M. Bachtiar, "Architecture design of low level control omni directional robot with rtos-rtx arm cortex-m4," in *2016 Electrical Power, Electronics, Communications, Controls and Informatics Seminar (ELECSYM)*, 09 2016, pp. 191–196.

[11] *nRF24L01 Single Chip 2.4GHz Transceiver*, Nordic Semicondutor, Julho 2007, rev 2.

[12] G. F. Franklin, J. D. Powell, and M. L. Workman, *Digital Control of Dynamic Systems*. USA: Ellis-Kagle Press, 1997.

[13] E. H. Currie, *Introduction to Embedded System*. Cham: Springer International Publishing, 2021, pp. 1–34.

[14] R. Barry, *Mastering the FreeRTOS™ Real Time Kernel*. EUA: Real Time Engineers Ltd., 2016.

[15] F. C. Reis Pinheiro, M. Ricardo Omena de Albuquerque Máximo, and T. Yoneyama, "Model predictive control for omnidirectional small size robot with motor and non-slipping constraints," in *2019 Latin American Robotics Symposium (LARS), 2019 Brazilian Symposium on Robotics (SBR) and 2019 Workshop on Robotics in Education (WRE)*, 2019, pp. 13–18.

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on July 31,2026 at 16:34:22 UTC from IEEE Xplore. Restrictions apply.